



PROGRAMAÇÃO



Escola Naval



António Gonçalves

Agenda

Dicionários: pares chave–valor

Consulta e atualização.

Chave	Valor
nome	Ana
idade	20
curso	Informática

Consultar
e atualizar
informação.

Métodos e processamento

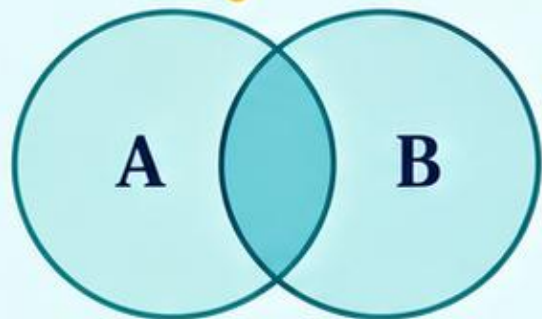
Percorrer, contar, combinar e filtrar.



Percorrer dados.
Contar elementos.
Combinar informação.
Filtrar resultados.

Conjuntos

Elementos únicos e operações entre grupos.



Elementos únicos.
Operações entre grupos.
Comparação de dados.

Persistência e aplicação

Guardar dados e gerir inscrições.



Guardar dados.
Gerir inscrições.
Aplicar os conceitos
num problema real.



Exercício final: gestão de alunos.

DICIONÁRIOS: PARES CHAVE-VALOR

Um **dicionário** é uma coleção mutável de pares chave-valor. As chaves são únicas e os valores podem ser de qualquer tipo.

Exemplo:

```
alunos = {101: 'Ana', 102: 'Bruno', 103: 'Carla'}
```

CHAVE

101

102

103

VALOR

'Ana'

'Bruno'

'Carla'

A **chave** identifica o valor associado.

Cada chave é única e permite aceder rapidamente ao valor correspondente.

Aceder a um elemento numa lista:

```
nomes[0]
```

Devolve o primeiro elemento da lista.

Aceder a um valor num dicionário:

```
alunos[101]
```

Devolve o nome do aluno com chave 101.

Número de pares no dicionário:

```
len(alunos)
```

Devolve 3.

A chave identifica o valor.

Criação e tipos de chaves

Criação de um dicionário

Um dicionário é uma coleção de pares chave-valor, escritos entre chavetas.

```
produto = {'nome': 'Teclado',  
           'preco': 29.90,  
           'stock': 12}
```

CHAVE	VALOR	TIPO
nome	Teclado	str
preco	29.90	float
stock	12	int

Dicionário vazio

Pode ser criado com chavetas vazias ou com a função `dict()`.

```
{}  
dict()
```

Tipos de chaves

As chaves têm de ser hashable. Exemplos aceites:

- ✓ `str` (cadeias de caracteres)
- ✓ `int` (números inteiros)
- ✓ `tuplos` (de elementos hashable)

Não são aceites como chaves:

- ✗ `list`
- ✗ `dict`
- ✗ `set`

Os valores podem repetir

Diferentes chaves podem ter o mesmo valor.

```
pessoas = {'Ana': 20,  
           'Bruno': 20}
```

Os valores podem repetir.

Chave repetida

Se a mesma chave for atribuída mais do que uma vez, o último valor sobrepõe os anteriores.

```
idades = {'Ana': 16, 'Ana': 18}
```



```
{'Ana': 18}
```

Fica o último valor atribuído à chave.

Consulta segura

```
notas = {'Ana': 16, 'Bruno': 13}
```

Dicionário que associa nomes de alunos às respectivas notas.



- O método **get** não insere a chave no dicionário.
- O valor por omissão é usado apenas quando a chave não existe.

```
nome = input('Nome: ').strip()
if nome in notas:
    print(notas[nome])
else:
    print('Aluno não encontrado.')
```

Expressão	Resultado	Explicação
<code>notas['Ana']</code>	16	Devolve a nota da aluna 'Ana'.
<code>notas['Carla']</code>	KeyError	Gera um erro porque a chave não existe.
<code>notas.get('Carla')</code>	None	Devolve None se a chave não existe.
<code>notas.get('Carla', 0)</code>	0	Devolve o valor por omissão apenas se a chave não existe.
<code>'Ana' in notas</code>	True	Testa se a chave 'Ana' existe.
<code>16 in notas</code>	False	Procura a chave 16, que não existe.

Lê o nome de um aluno, remove espaços em branco e mostra a respectiva nota. Se o nome não existir, exibe uma mensagem.

Chaves	Valores
'Ana'	16
'Bruno'	13

Dicionário notas

Adicionar, atualizar e remover

Adicionar um elemento

Atribuir um valor a uma nova chave adiciona o par ao dicionário.

```
notas = {'Ana': 16, 'Bruno': 13}
notas['Carla'] = 18
```

notas	
'Ana'	16
'Bruno'	13



notas	
'Ana'	16
'Bruno'	13
'Carla'	18

Atualizar um valor

Atribuir um valor a uma chave existente substitui o valor anterior.

```
notas = {'Ana': 16, 'Bruno': 13}
notas['Bruno'] = 15
```

notas	
'Ana'	16
'Bruno'	13



notas	
'Ana'	16
'Bruno'	15

Remover com del

A instrução **del** remove o par com a chave indicada. Se a chave não existir, ocorre `KeyError`.

```
notas = {'Ana': 16, 'Bruno': 13}
del notas['Bruno']
```

notas	
'Ana'	16
'Bruno'	13



notas	
'Ana'	16

Se a chave não existir, é lançada a exceção `KeyError`.

Remover com pop

O método `pop` remove o par com a chave indicada e devolve o respetivo valor. Se a chave não existir, devolve o valor por omissão (`None`).

```
notas = {'Ana': 16, 'Bruno': 13}
valor = notas.pop('Bruno', None)
```

notas	
'Ana'	16
'Bruno'	13



notas	
'Ana'	16

valor é 13
se a chave existir
ou `None`
se não existir.

Esvaziar o dicionário

O método `clear` remove todos os pares do dicionário.

```
notas = {'Ana': 16, 'Bruno': 13}
notas.clear()
```

notas	
'Ana'	16
'Bruno'	13



notas	
-------	--

Remover o último elemento

O método `popitem` remove e devolve o último par inserido no dicionário. Se o dicionário estiver vazio, ocorre `KeyError`.

```
notas = {'Ana': 16, 'Bruno': 13}
notas.popitem()
```

notas	
'Ana'	16
'Bruno'	13



notas	
'Ana'	16

Devolve o par
(`'Bruno'`, `13`).
Se estiver vazio,
é lançada
a exceção
`KeyError`.

Percorrer chaves, valores e pares

Dicionário: notas={'Ana':16, 'Bruno':13, 'Carla':18}

Percorrer as chaves

```
for nome in notas:  
    print(nome)
```



```
Ana  
Bruno  
Carla
```

notas.keys(): devolve uma vista com as chaves do dicionário.

Percorrer os valores

```
for nota in notas.values():  
    print(nota)
```



```
16  
13  
18
```

notas.values(): devolve uma vista com os valores do dicionário.

Percorrer os pares

```
for nome, nota in notas.items():  
    print(nome, nota)
```



```
Ana 16  
Bruno 13  
Carla 18
```

notas.items(): devolve uma vista com os pares (chave, valor) do dicionário, na forma de tuplos.



A ordem de iteração é a **ordem de inserção** das chaves, não há ordenação alfabética automática.



Não adicionar nem remover chaves durante a iteração.

MÉDIAS E FUNÇÕES

Dicionário de notas:

associa o nome de cada aluno à respetiva nota.

Verificação de dicionário vazio:

se não existirem notas, evita a divisão por zero e devolve None.

Devolução do resultado:

a instrução `return` permite reutilizar a média noutras partes do programa.

```
notas={ 'Ana':15, 'Pedro':12, 'Carla':18}

def calcular_media(notas):
    if not notas:
        return None
    return sum(notas.values()) / len(notas)

media = calcular_media(notas)
if media is not None:
    print(media)
```

Execução e resultado:

o programa calcula a média das notas e apresenta:

15.0

O valor devolvido pela função é guardado em `media` e depois utilizado na condição.

Outra função:

Verifica se o número existe como chave no dicionário de alunos.

```
def tem_aluno(alunos, numero):
    return numero in alunos
```

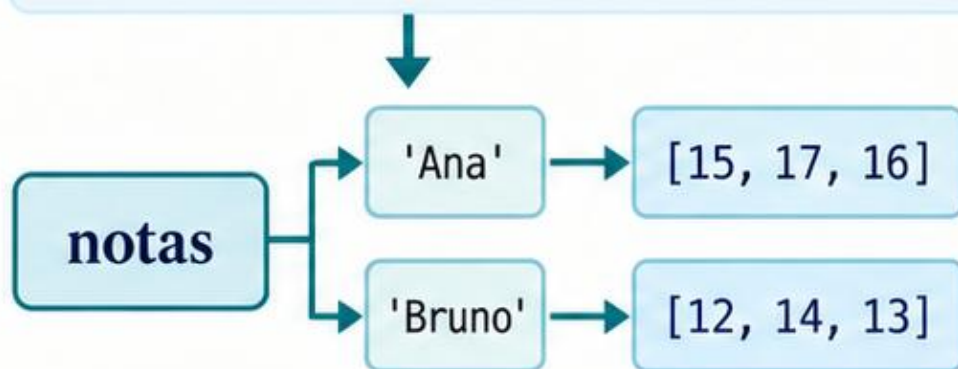
Operador in:

Verifica se a chave `numero` existe em `alunos` e devolve `True` ou `False`.

Dicionários de listas e tuplos

Um dicionário pode ter listas como valores. Podemos percorrer os pares chave-valor, calcular médias e também iterar sobre as listas.

```
notas = {'Ana': [15, 17, 16], 'Bruno': [12, 14, 13]}
for nome, valores in notas.items():
    if valores:
        print(nome, sum(valores) / len(valores))
```



Resultado:

Ana 16.0
Bruno 13.0

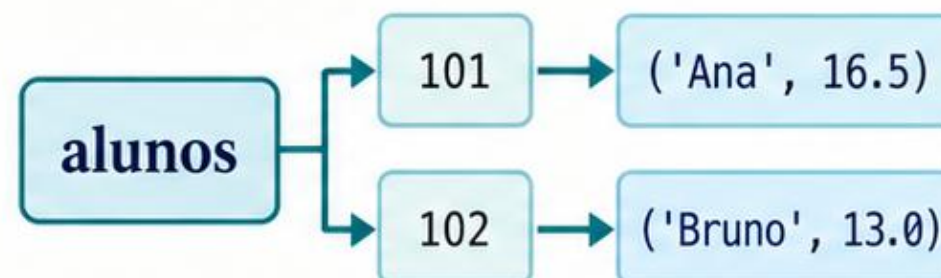
```
for valores in notas.values():
    for nota in valores:
        print(nota)
```

Resultado:

15
17
16
12
14
13

Um dicionário pode ter tuplos como valores. Podemos aceder a um elemento do dicionário e decompor o tuplo em variáveis.

```
alunos = {101: ('Ana', 16.5), 102: ('Bruno', 13.0)}
nome, nota = alunos[101]
print('nome', nome, 'nota', nota)
```



Resultado:

nome Ana nota 16.5

Os valores de um dicionário podem ser coleções estruturadas, como listas ou tuplos.

Combinação e compreensão

Combinação Python3.9+:

```
base={'tema':'claro','idioma':'pt'}  
extra={'tema':'escuro'}  
novo=base | extra
```



Resultado: {'tema':'escuro','idioma':'pt'}

base { 'tema': 'claro', 'idioma': 'pt' }	+	extra { 'tema': 'escuro' }	→	novo { 'tema': 'escuro', 'idioma': 'pt' }
---	----------	--	----------	--

`base |= extra` actualiza base.
`base.update(extra)` é uma alternativa.
Em chaves duplicadas, o valor da direita prevalece.

Compreensão:

```
notas={'Ana':16,'Bruno':8,'Carla':18}  
aprovados={nome: nota for nome, nota in notas.items()  
          if nota >= 10}
```



Resultado: {'Ana':16,'Carla':18}

Cria um novo dicionário com os alunos que têm nota maior ou igual a 10.

```
quadrados={n:n**2 for n in range(1,4)}
```



Resultado: {1:1,2:4,3:9}

Cria um dicionário com os quadrados dos números de 1 a 3.

Frequência de palavras

Conta quantas vezes cada palavra aparece num texto.

Exemplo

```
frase = 'python dados python listas dados python'
frequencias = {}
for palavra in frase.lower().split():
    frequencias[palavra] = frequencias.get(palavra, 0) + 1
print(frequencias)
```

Resultado

```
{'python': 3, 'dados': 2, 'listas': 1}
```

Contagens de palavras

Palavra	Contagem
python	3
dados	2
listas	1

1 Inicialização

O dicionário `frequencias` começa vazio. Quando uma palavra ainda não existe, `frequencias.get(palavra, 0)` retorna `0`.

2 Separação das palavras

O método `split()` divide o texto em palavras usando os espaços em branco. A pontuação não é removida automaticamente.

3 Atualização da contagem

A contagem aumenta uma unidade por ocorrência.

4 Ficheiros de texto

Num ficheiro de texto, percorrer as linhas e, dentro de cada linha, percorrer as palavras.

CONJUNTOS: ELEMENTOS ÚNICOS

Um **conjunto** (*set*) é uma coleção de elementos únicos e *hashable*.

É mutável, não tem indexação e não garante uma ordem de iteração.

Exemplo: elementos únicos

```
>>> valores = [10, 20, 10, 30, 20]
>>> unicos = set(valores)
>>> unicos
{10, 20, 30}
>>> len(unicos)
3
```

Lista com repetidos

[10, 20, 10, 30, 20]



set(valores)



Conjunto
(elementos únicos)
{10, 20, 30}

A ordem dos elementos é arbitrária.

Conjunto vazio

```
>>> vazio = set()
>>> vazio
set()
```

Usa-se **set()** para criar um conjunto vazio. **{}** cria um dicionário vazio, não um conjunto.

A partir de uma cadeia de caracteres

```
>>> set('banana')
{'b', 'a', 'n'}
```

Cada carácter único da cadeia torna-se um elemento do conjunto.

A partir de uma tupla

```
>>> set((1, 2, 2, 3))
{1, 2, 3}
```

Elementos repetidos são removidos automaticamente.

Sem indexação

```
>>> unicos[0]
Traceback (most recent call last):
...
TypeError: 'set' object is not subscriptable
```

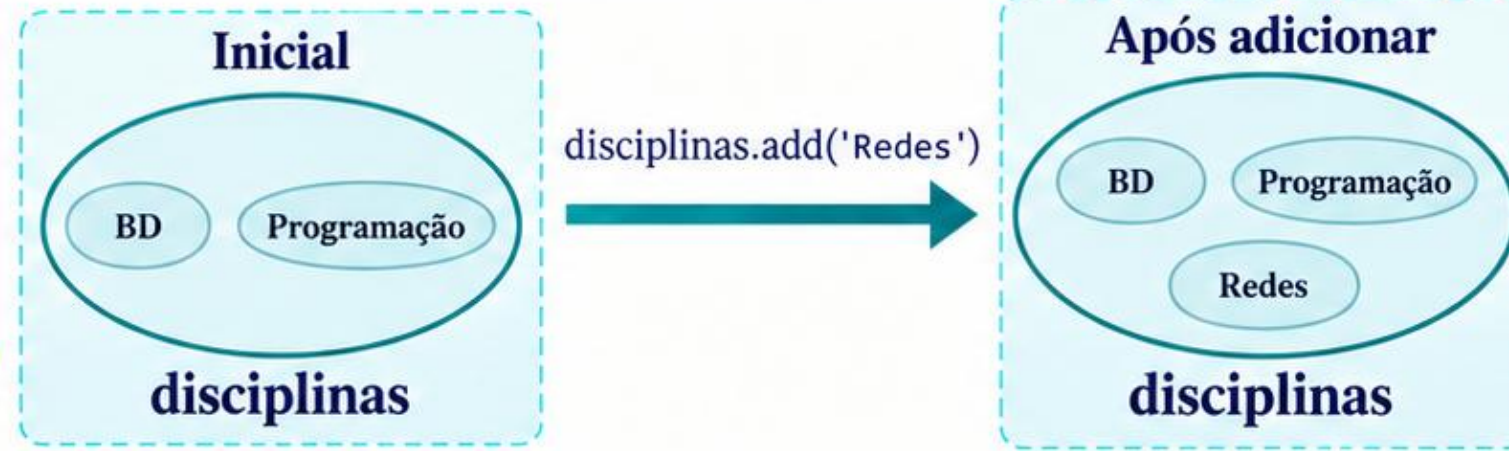
Os conjuntos não suportam indexação por posição.

Tipos de elementos

Listas, dicionários e conjuntos não podem ser elementos de um conjunto, porque não são *hashable*.

ADICIONAR E REMOVER NUM SET

Um **set** é uma coleção não ordenada de elementos únicos. Podemos adicionar e remover elementos usando métodos próprios.



Adicionar um elemento que já existe não altera o **set**, porque os elementos são únicos.

```
disciplinas = {'BD', 'Programação'}
disciplinas.add('Redes')
disciplinas.add('BD') # já existe
print(len(disciplinas)) # agora 3
3
```

```
disciplinas.update(['IA', 'Redes'])
print(len(disciplinas)) # agora 4
4
```

Método	Descrição
add(x)	Adiciona um único elemento ao set .
update(iteravel)	Adiciona vários elementos (de um iterável).
remove(x)	Remove um elemento. Lança um erro <code>KeyError</code> se o elemento não existir.
discard(x)	Remove um elemento. Não lança erro se o elemento não existir.
clear()	Remove todos os elementos, ficando o set vazio.

Os operadores **in** e **not in** permitem verificar se um elemento pertence a um **set**.

'BD' **in** disciplinas

'Redes' **not in** disciplinas

UNIÃO E INTERSEÇÃO

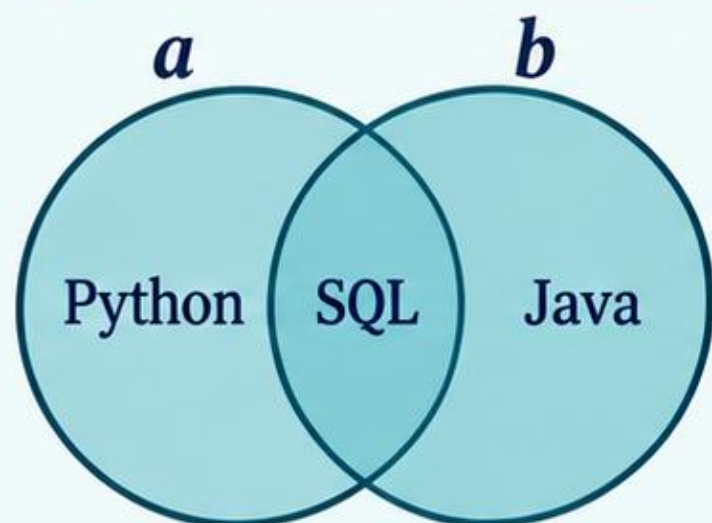
```
a = {'Python', 'SQL'}  
b = {'SQL', 'Java'}
```

As operações de união e interseção resultam em novos conjuntos e não alteram os conjuntos originais.

União (|)

Elementos que estão em pelo menos um dos conjuntos.

```
a.union(b)
```



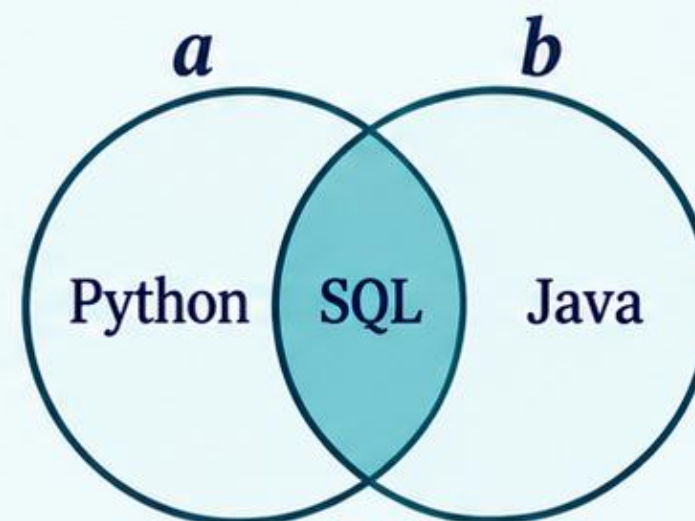
```
{ 'Python',  
  'SQL',  
  'Java' }
```

Todos os elementos de ambos os conjuntos (sem repetições).

Interseção (&)

Elementos que estão em ambos os conjuntos.

```
a.intersection(b)
```



```
{ 'SQL' }
```

Elementos comuns aos dois conjuntos.

Os conjuntos originais a e b mantêm-se inalterados.

Diferenças entre conjuntos

$a = \{\text{'Python'}, \text{'SQL'}\}$
 $b = \{\text{'SQL'}, \text{'Java'}\}$

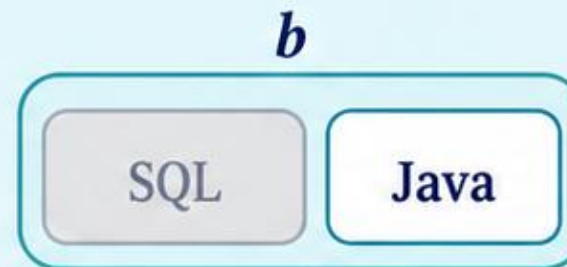
`a.difference(b)`
`b.difference(a)`
`a.symmetric_difference(b)`

Elementos de a
que não estão em b
 $a - b$



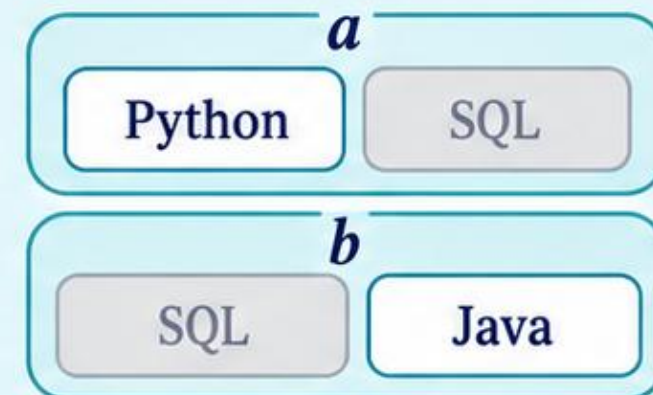
$a - b = \{\text{'Python'}\}$

Elementos de b
que não estão em a
 $b - a$



$b - a = \{\text{'Java'}\}$

Elementos que estão
em apenas um dos conjuntos
 $a \wedge b$



$a \wedge b = \{\text{'Python'}, \text{'Java'}\}$

Diferença de conjuntos: é uma operação direcional. $a - b$ contém elementos que estão em a mas não estão em b , e $b - a$ o contrário.

Diferença simétrica: $a \wedge b$ contém elementos que estão em apenas um dos conjuntos, excluindo os elementos comuns (neste caso, 'SQL').

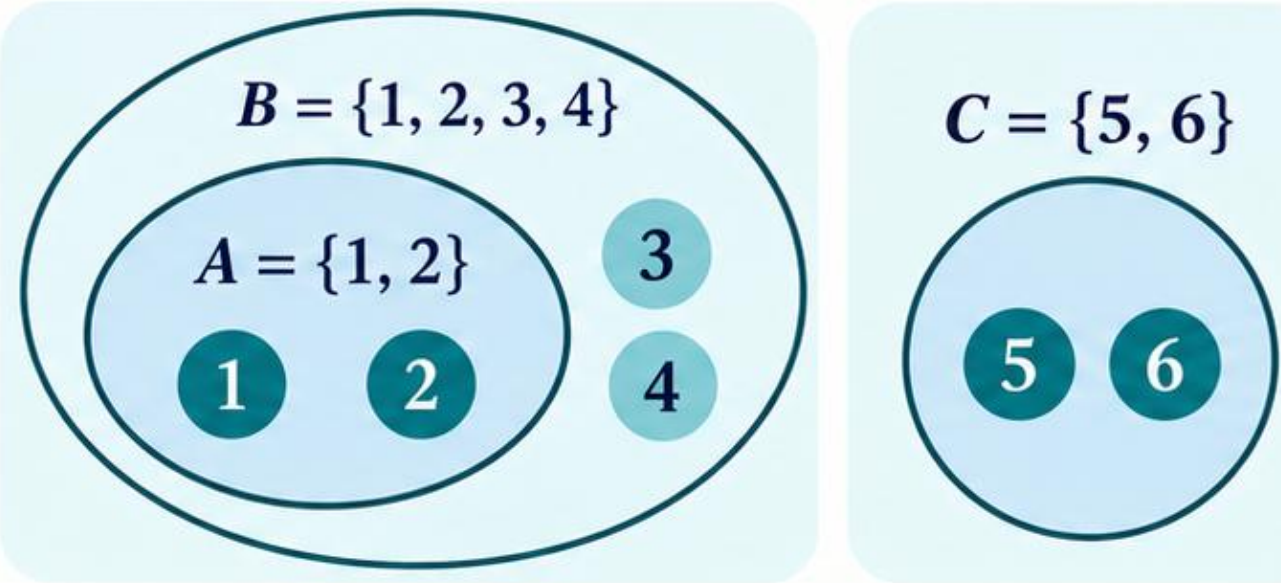
O conjunto a e o conjunto b têm o elemento comum 'SQL'.

A diferença simétrica $a \wedge b$ exclui os elementos comuns e inclui apenas 'Python' e 'Java'.

Inclusão e conjuntos disjuntos

Um conjunto A está contido num conjunto B se todos os elementos de A pertencem a B .

Se A está contido em B e $A \neq B$, então A é um **subconjunto próprio** de B .



Dois conjuntos são **disjuntos** se não têm elementos comuns.

A comparação de inclusão considera os **elementos** dos conjuntos e não o número de elementos.

Código em Python:

```
a = {1, 2}
b = {1, 2, 3, 4}
c = {5, 6}

a.issubset(b)
b.issuperset(a)
```

Expressão	Significado	Resultado
$a \leq b$	A é subconjunto de B .	True
$a < b$	A é subconjunto próprio de B .	True
$b \geq a$	B é superconjunto de A .	True
$a \leq a$	A é subconjunto de A .	True
$a < a$	A é subconjunto próprio de A .	False
<code>a.isdisjoint(c)</code>	A e C são disjuntos.	True
<code>a.issubset(b)</code>	Equivalente a $a \leq b$.	True
<code>b.issuperset(a)</code>	Equivalente a $b \geq a$.	True

Observações:

- 1 A comparação de inclusão verifica se todos os elementos de um conjunto pertencem ao outro. Não depende apenas do número de elementos.
- 2 Conjuntos disjuntos não têm elementos em comum, como A e C neste exemplo.

Iteração e compreensão de sets

Um **set** é uma coleção não ordenada de elementos **únicos**. Podemos iterar sobre um **set**, ordenando os elementos para obter uma saída previsível.

```
nomes = ['Ana', 'Bruno', 'Ana', 'Carla', 'Bruno']
unicos = set(nomes)
for nome in sorted(unicos):
    print(nome)
```

lista original
(com repetidos)

['Ana', 'Bruno',
'Ana', 'Carla',
'Bruno']

set
(elementos únicos)

{'Ana', 'Bruno',
'Carla'}

saída

Ana
Bruno
Carla

A função **sorted** devolve uma lista com os elementos do **set**, permitindo uma ordem previsível na iteração.

Podemos criar **sets**, usando compreensões de conjuntos para selecionar ou transformar elementos de uma sequência.

```
numeros = [1, 2, 2, 3, 4, 5, 6]
pares = {n for n in numeros if n % 2 == 0}
quadrados = {n**2 for n in numeros}
```

lista original

[1, 2, 2, 3, 4, 5, 6]

pares

($n \% 2 == 0$)

{2, 4, 6}

quadrados

($n**2$)

{1, 4, 9, 16, 25, 36}

Os **sets** não têm uma ordem garantida, pelo que a ordem de apresentação dos elementos pode variar.

Também podemos usar **sets** para encontrar palavras únicas numa frase, ignorando maiúsculas/minúsculas.

```
frase = 'o rato roeu a rolha do rei'
len(set(frase.lower().split()))
```

frase

'o rato roeu a rolha do rei'

palavras (minúsculas)

['o', 'rato', 'roeu', 'a', 'rolha', 'do', 'rei']

set (palavras únicas)

{'o', 'rato', 'roeu', 'a', 'rolha', 'do', 'rei'}

número de palavras únicas

7

Dicionários e sets no mesmo programa

Dicionário de alunos

```
alunos={101:'Ana',102:'Bruno',103:'Carla',104:'Diogo'}
```

Um dicionário associa chaves a valores. Aqui, cada número de aluno está associado ao respetivo nome.

Conjuntos de disciplinas

```
python={101,102,103}  
bd={102,104}
```

Os sets guardam elementos e permitem operar com conjuntos de forma simples.



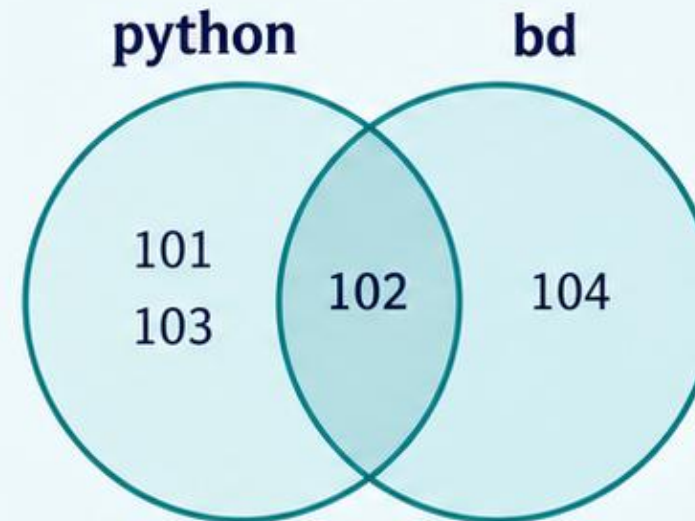
Os dicionários armazenam dados.



Os sets guardam informação de pertença.

Operações com sets

Operação	Resultado
ambos (python & bd)	{102}
só Python (python - bd)	{101,103}
pelo menos uma (python bd)	{101,102,103,104}
só uma (python ^ bd)	{101,103,104}



Percorrer a interseção

```
for numero in sorted(python & bd):  
    print(numero, alunos[numero])
```

Saída do programa

```
102 Bruno
```



Os dicionários armazenam dados.



Os sets guardam informação de pertença.

Persistência com pickle

Serializar transforma um objeto Python em bytes. **Desserializar** recupera o objeto.

Código em Python:

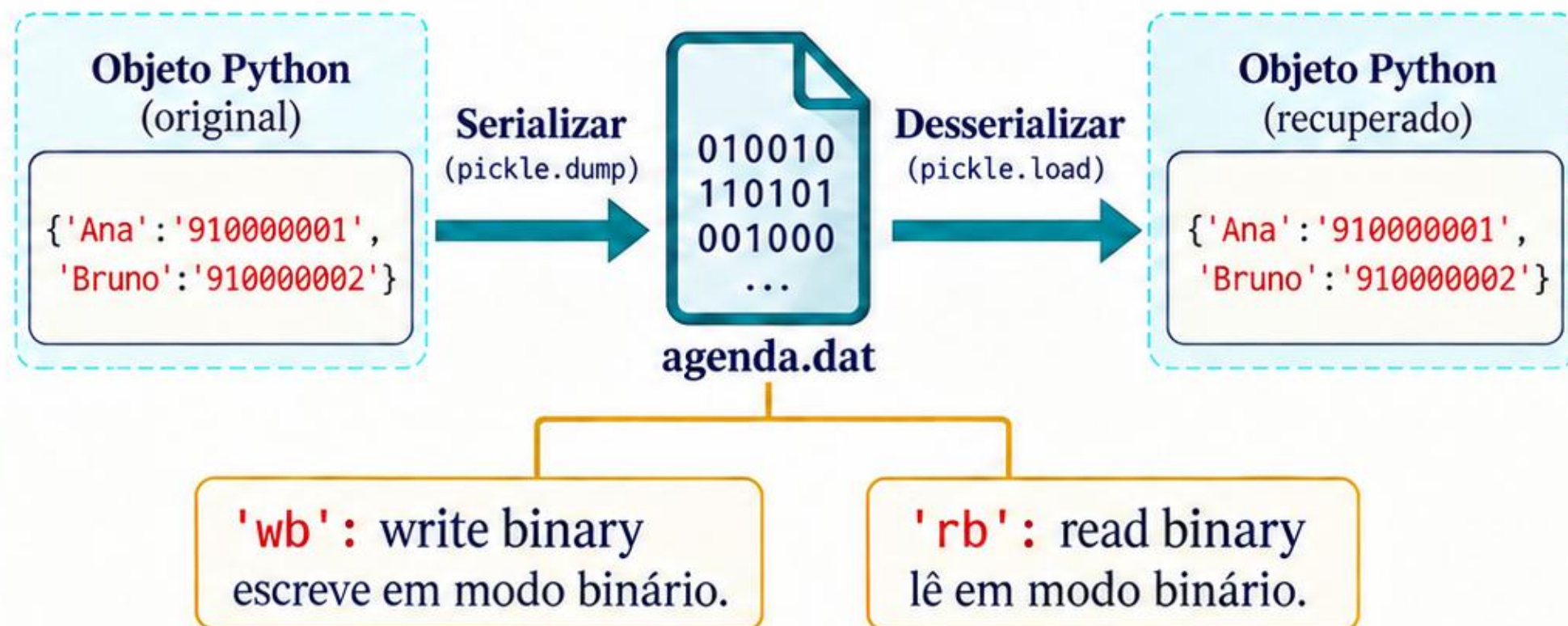
```
import pickle
agenda={'Ana':'910000001','Bruno':'910000002'}
with open('agenda.dat', 'wb') as f:
    pickle.dump(agenda, f)

with open('agenda.dat', 'rb') as f:
    recuperada=pickle.load(f)

print(recuperada['Ana'])
```

Saída:

910000001



Ler ficheiros pickle pode executar código.
Carregar apenas ficheiros de origem confiável.

Funções para guardar e carregar

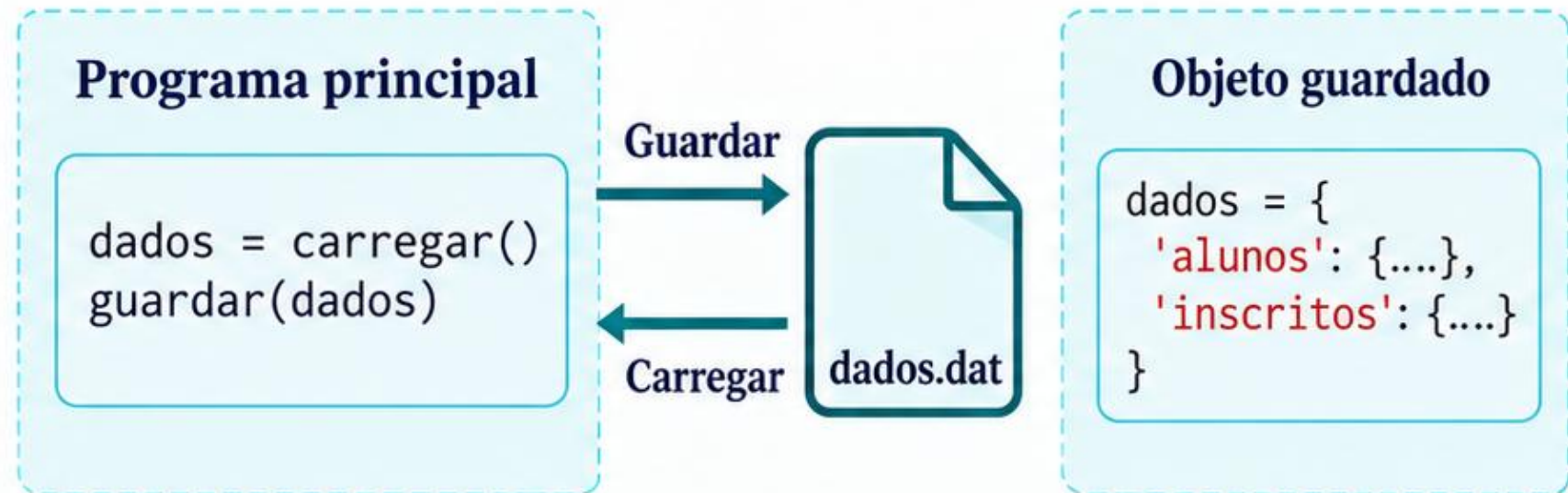
```
import pickle

FICHEIRO = 'dados.dat'

def guardar(dados):
    with open(FICHEIRO, 'wb') as f:
        pickle.dump(dados, f)

def carregar():
    try:
        with open(FICHEIRO, 'rb') as f:
            return pickle.load(f)
    except FileNotFoundError:
        return {'alunos': {}, 'inscritos': set()}
```

As funções guardam e carregam um objeto que contém um dicionário (**dict**) e um conjunto (**set**).



1 Ficheiro em falta

Se o ficheiro não existir, a função `carregar()` inicia com um objeto vazio: `{'alunos': {}, 'inscritos': set()}`.

2 Outros erros

Outros erros não são ignorados silenciosamente e são propagados.

3 Dados confiáveis

Utilizar `pickle` apenas com ficheiros de origem confiável.

4 Organização

As funções podem ser mantidas num ficheiro `dados.py` e importadas pelo programa principal.

Registo e inscrição de alunos

Função adicionar

Adiciona um aluno ao dicionário.

```
def adicionar(alunos, numero, nome):  
    nome = nome.strip()  
    if numero in alunos or not nome:  
        return False  
    alunos[numero] = nome  
    return True
```

Rejeita identificadores duplicados e nomes vazios (após remover espaços em branco).

Função inscrever

Inscreve um aluno num conjunto de inscritos.

```
def inscrever(alunos, inscritos, numero):  
    if numero not in alunos:  
        return False  
    inscritos.add(numero)  
    return True
```

Só inscreve alunos que existem no dicionário. O conjunto evita inscrições duplicadas.

Dicionário de alunos (alunos)

Número	Nome
101	Ana
...	...

Conjunto de inscritos (inscritos)

{ 101 }

...

Exemplos de utilização

```
alunos = {}  
inscritos = set()  
adicionar(alunos, 101, "Ana")  
# True e alunos = {101: 'Ana'}  
inscrever(alunos, inscritos, 101)  
# True e inscritos = {101}  
inscrever(alunos, inscritos, 999)  
# False (aluno não existe)
```

Manter a consistência

Ao remover um aluno do dicionário, deve também ser removida a sua inscrição:

```
inscritos.discard(numero)
```

Assim, o conjunto mantém-se consistente com o dicionário.

Estatísticas e visualização

Calcular a média das notas:

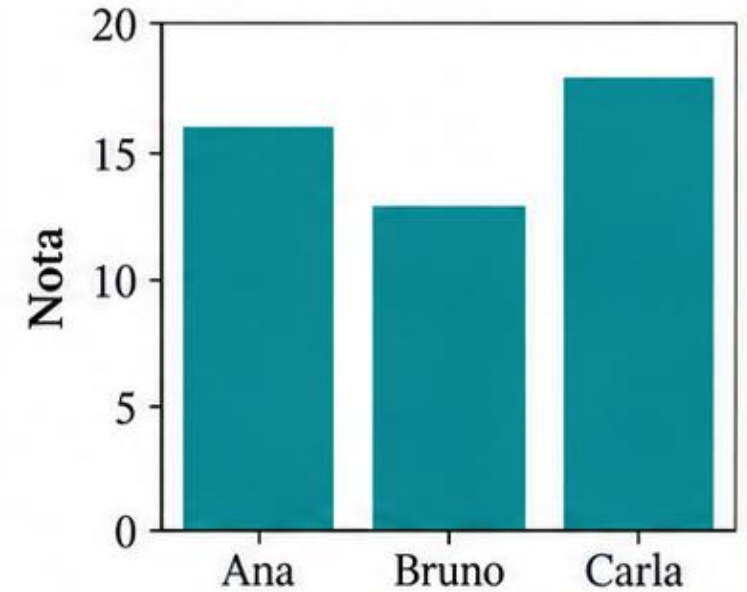
```
notas={'Ana':16,'Bruno':13,'Carla':18}  
print(sum(notas.values()) / len(notas))
```

Resultado: 15.666...

É necessário um tratamento para o caso de dicionário vazio antes de calcular a média.

Visualizar as notas num gráfico de barras:

```
import matplotlib.pyplot as plt  
plt.bar(list(notas.keys()),  
        list(notas.values()))  
plt.ylabel('Nota')  
plt.ylim(0, 20)  
plt.show()
```



Exemplo com valores aleatórios:

```
import random  
teste={n: random.randint(0,20) for n in range(101,106)}
```

Cria um dicionário com cinco identificadores (101 a 105) e notas aleatórias entre 0 e 20 (inclusive).

ID	Nota
101	...
102	...
103	...
104	...
105	...

Exercício: gestão de alunos

Dados iniciais

```
alunos = {101: 'Ana', 102: 'Bruno',  
          103: 'Carla', 104: 'Diogo'}  
python = {101, 102, 103}  
bd = {102, 104}
```



Gestão de alunos



Funcionalidades a implementar

- 1 Adicionar aluno
- 2 Pesquisar aluno
- 3 Editar aluno
- 4 Remover aluno
- 5 Validar identificador e nome não vazio
- 6 Inscrever apenas alunos registados
- 7 Comparar atividades
- 8 Guardar e carregar um único objeto com pickle (ambiente de confiança)
- 9 Tratar ficheiro em falta

Resultados esperados

Alunos inscritos em ambas as atividades (Python e BD):

102 Bruno

Alunos apenas em Python:

101 Ana

103 Carla

Total de alunos únicos inscritos: 4

Ações do menu (rótulos)

- 1 Adicionar
- 2 Pesquisar
- 3 Editar
- 4 Remover
- 5 Inscrever em Python
- 6 Inscrever em BD
- 7 Comparar atividades
- 8 Guardar
- 9 Carregar
- 0 Sair

Casos de teste

- 1 Tentativa de adicionar um aluno duplicado (identificador 101).
- 2 Pesquisa de um aluno desconhecido (identificador 999).
- 3 Tentativa de adicionar um aluno com nome vazio.
- 4 Tentativa de inscrever um aluno que já está inscrito.
- 5 Carregamento com ficheiro em falta.

Reutilizar as funções guardar e carregar apresentadas anteriormente.

Guardar os alunos e os dois conjuntos de inscrições no mesmo objeto.

Carregar apenas ficheiros de origem confiável.

Solução base e síntese

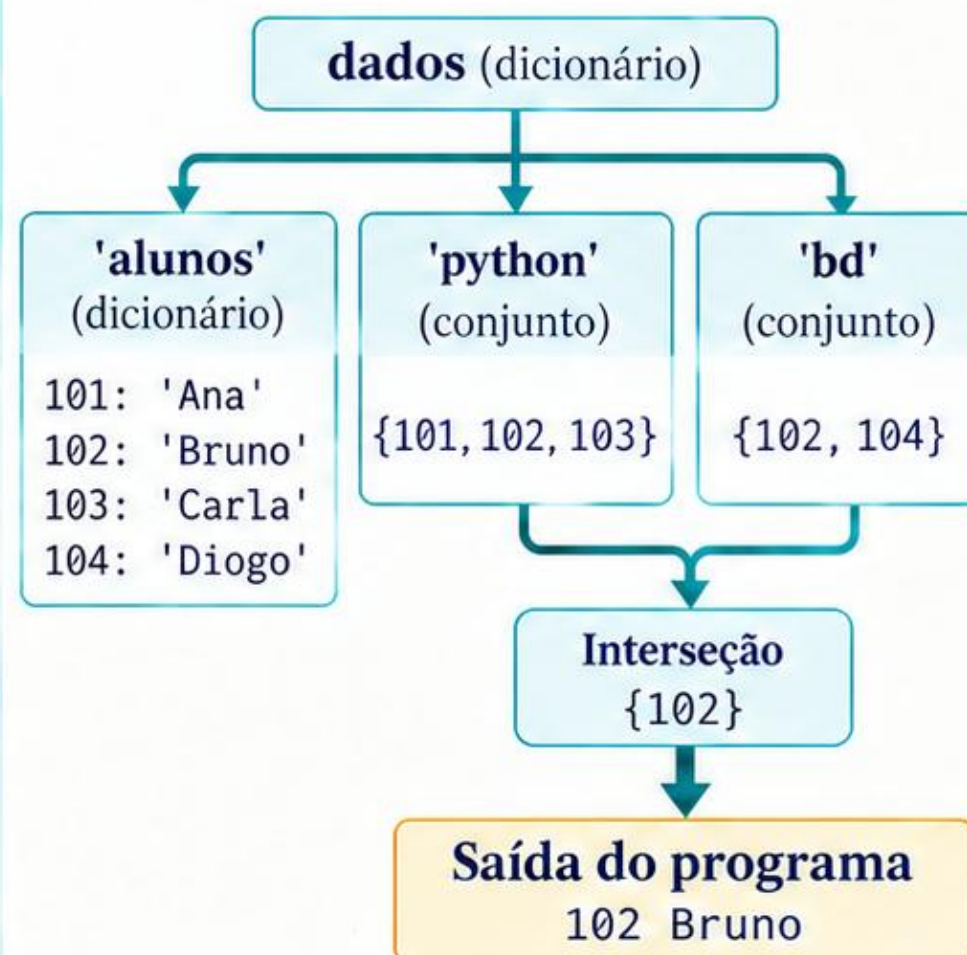
Código de exemplo

```
dados = {  
    'alunos': {101: 'Ana', 102: 'Bruno',  
               103: 'Carla', 104: 'Diogo'},  
    'python': {101, 102, 103},  
    'bd': {102, 104}  
}
```

```
comuns = dados['python'] & dados['bd']  
for numero in sorted(comuns):  
    print(numero, dados['alunos'][numero])
```

```
guardar(dados)
```

Estrutura dos dados



Persistência dos dados

Ação	Código
Guardar no ficheiro	guardar(dados)
Recuperar do ficheiro	dados = carregar()

Na função carregar, adaptar o valor inicial às chaves alunos, python e bd.

```
{'alunos': {},  
 'python': set(),  
 'bd': set()}
```

se o ficheiro não existir nesta tarefa.

Conceitos principais

- 1 Dicionário:** associa dados através de chaves.
- 2 Conjunto:** permite verificar pertença e operações entre conjuntos (por exemplo, interseção com &).
- 3 Pickle:** permite persistir os dados em ficheiro.

Exercícios

Estender as funções desenvolvidas anteriormente.